

Rust

Notas del estudio

Giovanny Alfonso Chávez Ceniceros

Giovanny Alfonso Chávez Ceniceros

Rust

Notas de estudio

Contenido

1. Sintaxis y Semántica

- 1.1 Tokens
- 1.2 Tipos de Dato
- 1.3 Estructuras de Control
- 1.4 Tipos de Memoria
- 1.5 Funciones

2. Memory Safety

- 2.1 Ownership
- 2.2 Borrowing y Lifetimes

Sintaxis y Semántica

Tokens

Palabras reservadas (*Keywords*)

1. Son la columna vertebral de la sintaxis de Rust.
 2. Dictan la definición de los bloques fundamentales de construcción de código.
 3. Omitir o usar incorrectamente alguna de ellas resulta en un error en tiempo de compilación.
- `let` se usa para la declaración de variables (auxiliada por `mut` para permitir la mutabilidad).
 - `fn` se usa para la definición de funciones (auxiliada por `return` para devolver valores explícitamente).
 - `if` se usa para condiciones (junto con `else` y `match` para bifurcar el flujo del programa).
 - `loop` , `while` y `for` se usan para ciclos (donde `in` auxilia a los ciclos `for`).
 - Existen muchas más (como `self` , `struct` , `enum` , `trait` , `impl` , `mod` , `type`), las cuales tienen nichos de uso más específicos o avanzados.

Identificadores

1. Son la parte personalizable del programa.
2. Es el nombre que el desarrollador asigna a un recurso (como una variable o función) para poder invocarlo más adelante.
3. Seguir las convenciones de nomenclatura de Rust (*snake_case*) y mantener nombres descriptivos mejora significativamente la legibilidad y el mantenimiento del código.

Literales

1. Son valores fijos que se incrustan directamente en el código fuente (como `42` , `"hola"` o `true`).
2. Son inmutables por naturaleza y ofrecen ventajas de rendimiento, ya que el compilador los conoce desde el principio.

Operadores

1. Son símbolos que indican al compilador la ejecución de tareas matemáticas, lógicas o de asignación.
 2. Actúan sobre variables y literales (llamados operandos) para producir un nuevo valor.
- `+` , `-` , `*` , `/` , `%` realizan operaciones aritméticas (suma, resta, multiplicación, división y residuo).
 - `==` , `!=` , `<` , `>` , `<=` , `>=` realizan comparaciones que devuelven un valor booleano (`true` o `false`).
 - `&&` , `||` , `!` ejecutan la lógica booleana (AND, OR y NOT).
 - `=` asigna valores, y puede combinarse con operadores aritméticos (como `+=` o `-=`) para crear asignaciones compuestas.

Puntuaciones

1. Son caracteres individuales que definen la estructura semántica y el ritmo del código.
 2. Aunque parecen sutiles, son estrictamente obligatorios para que el compilador entienda dónde termina una idea y empieza otra.
- El punto y coma `;` se usa para cerrar enunciados y expresiones de asignación.
 - Los dos puntos `:` se usan para especificar o anotar tipos de datos.

Delimitadores

1. Son caracteres que definen la estructura semántica del código respecto a los **ámbitos (scopes)** y a las secuencias de datos.
2. **Siempre trabajan en pares** (apertura y cierre) para aislar o agrupar información.
 - **Llaves** `{ }` : Definen un bloque de código y crean un ámbito (*scope*) aislado; todo lo declarado dentro muere al cerrarse la llave y no es accesible desde fuera.
 - **Corchetes** `[ ]` : Se utilizan para declarar arreglos (arrays) y para acceder a sus elementos mediante índices (ej. `let arr = [1, 2, 3];` y `arr[0]`).
 - **Paréntesis** `( )` : Encapsulan parámetros y argumentos en la declaración e invocación de funciones, y agrupan operaciones matemáticas para definir su prioridad.

Tipos de Dato

Categoría	Tipo	Descripción	Ejemplo / Sintaxis
Numéricos	i8 , i16 , i32 , i64 , i128 , isize	Enteros con signo (complemento a 2).	<pre>let x: i32 = -42; let ptr: isize = -1;</pre>
	u8 , u16 , u32 , u64 , u128 , usize	Enteros sin signo.	<pre>let x: u8 = 255; let idx: usize = 0;</pre>
	f32 , f64	Números de punto flotante según estándar IEEE 754.	<pre>let x: f64 = 3.1416; let y: f32 = 2.5;</pre>

Categoría	Tipo	Descripción	Ejemplo / Sintaxis
Texto	char	Representa un carácter puro de cualquier alfabeto, emoji o símbolo.	<pre>let c: char = 'z'; let emoji = '🦀';</pre>
	str	Cadena de texto fija. Se utiliza casi siempre como referencia inline (&str).	<pre>let s: &str = "hola";</pre>
	String	Cadena de texto dinámica, mutable, de tamaño variable.	<pre>let s = String::from("hola"); let s2 = "mundo".to_string();</pre>

Categoría	Tipo	Descripción	Ejemplo / Sintaxis
Lógicos y Vacíos	<code>bool</code>	Tipo booleano clásico de 1 byte de tamaño.	<pre>let activo: bool = true; let bandera = false;</pre>
	<code>()</code>	Equivale al concepto de <code>void</code> de otros lenguajes.	<pre>let vacio: () = (); fn log() -> () {}</pre>

Categoría	Tipo	Descripción	Ejemplo / Sintaxis
Colecciones	<code>[T; N]</code>	Arreglo (<i>Array</i>). Colección homogénea de longitud fija conocida en tiempo de compilación (<code>N</code>).	<code>let arr: [i32; 3] = [1, 2, 3];</code>
	<code>(T, U, ...)</code>	Tupla. Colección heterogénea de longitud fija.	<code>let t: (i32, &str, bool) = (42, "hola", true);</code>
	<code>[T]</code>	Secuencia contigua de tamaño dinámico de tipos <code>T</code> . Se opera mediante <code>&[T]</code> .	<code>let sub_arr: &[i32] = &arr[1..3];</code>

Categoría	Tipo	Descripción	Ejemplo / Sintaxis
Personalizados	struct	Estructuras. Permiten agrupar múltiples valores relacionados de distintos tipos bajo un mismo nombre.	<pre>struct Usuario { ... }</pre>

```
struct Person {  
    name: String,  
    age: u8,  
    is_dev: bool,  
}
```

```
fn main() {  
    let developer = Person {  
        name: String::from("Gio"),  
        age: 25,  
        is_dev: true,  
    };  
}
```

A diferencia de las tuplas, cada dato dentro de una `struct` tiene un **nombre claro (campo)**. Esto te permite modelar entidades del mundo real con un formato fijo que el compilador de Rust valida estrictamente.

Estructuras de Control

1. Condicional Estándar (`if` / `else`)

Evalúa una condición booleana pura. No lleva paréntesis en la condición.

```
if condicion_booleana {  
    // Se ejecuta si es true  
} else {  
    // Opcional: Se ejecuta si es false  
}
```

2. Coincidencia de Patrones Completa (`match`)

Obligatorio en Rust para evaluar múltiples caminos. Debe ser exhaustivo (cubrir todas las posibilidades).

```
match expresion {  
    patron_a => { /* código si coincide A */ },  
    patron_b => { /* código si coincide B */ },  
    _ => { /* opcional: "catch-all" para cualquier otro caso */ },  
}
```

3. Condicional de Patrón Único (`if let`)

La versión compacta de match cuando solo te importa un caso específico (ideal para extraer valores de Option o Result).

```
if let patron = expresion {  
    // Se ejecuta si coincidió; las variables internas del patrón se pueden usar aquí  
} else {  
    // Opcional: Se ejecuta si no coincidió  
}
```

4. Bucle por Colección (`for in`)

La forma más común y segura de iterar en Rust. Funciona con rangos, arreglos o vectores.

```
for variable in coleccion_o_rango {  
    // Se ejecuta por cada elemento (variable toma el valor actual)  
}
```

5. Bucle Condicional (`while`)

Ejecuta el bloque repetidamente mientras la condición booleana sea verdadera.

```
while condicion_booleana {  
    // Se ejecuta en bucle mientras sea true  
}
```

6. Bucle de Patrón Único (`while let`)

Ejecuta el bucle repetidamente mientras la expresión siga coincidiendo con el patrón (muy usado para vaciar colas o canales).

```
while let patron = expresion {  
    // Se ejecuta en bucle mientras siga coincidiendo el patrón  
}
```


7. Bucle Infinito (loop)

Un bucle eterno que no depende de una condición inicial. Es óptimo en Rust porque el compilador sabe que la única forma de salir es con un break explícito.

```
loop {  
    // Se ejecuta infinitamente  
    if condicion_de_salida {  
        break; // Detiene el bucle (puede devolver un valor: break valor;)  
    }  
}
```

Tipos de Memoria

El Stack

El Stack es una estructura de datos de acceso ultra rápido. Funciona bajo la regla de que lo último en llegar es lo primero en salir. El compilador solo permite guardar aquí datos cuyo tamaño exacto se conoce desde el momento en que se escribe el código.

- **¿Cómo funciona?:** Al llamar a una función, sus variables se "apilan" una arriba de otra en memoria. Al terminar la función, todo ese bloque se limpia instantáneamente.
- **Tipos que viven aquí:** Los Literales numéricos y booleanos, caracteres individuales (`char`), y las colecciones de tamaño fijo (Arreglos `[T; N]` y Tuplas).

- **Comportamiento:** Cuando una variable almacena un dato que vive completamente en el Stack, Rust permite que su valor se **duplique automáticamente** al asignarla a otra variable. Al hacer `let y = x;`, Rust clona los bits en la memoria de forma instantánea y transparente.
- **Resultado:** Ambas variables siguen vivas y son independientes. Esto ocurre porque duplicar datos pequeños y fijos en el Stack es sumamente barato para la computadora.

```
let x = 42;    // Vive en el Stack
let y = x;     // Se duplica automáticamente
println!("x: {}, y: {}", x, y); // ¡Ambos se pueden usar!
```

El Heap

El Heap se utiliza para datos más complejos cuyo tamaño es dinámico, desconocido al compilar, o que necesitan crecer y cambiar durante la ejecución del programa.

- **¿Cómo funciona?:** El sistema operativo busca un espacio libre en el Heap, guarda los datos ahí y genera un **apuntador** (una variable con la dirección de esa ubicación). El apuntador (que mide siempre lo mismo) se guarda en el Stack para acceso rápido, pero el contenido real está en el Heap.
- **Tipos que viven aquí:** Estructuras dinámicas como `String` o listas que pueden crecer.

- **Comportamiento:** A diferencia del Stack, los datos que se guardan en el Heap **no se duplican de forma automática**. Rust implementa una regla estricta de propiedad para evitar accidentes en la memoria. Al asignar una variable del Heap a otra (ej. `let s2 = s1;`), Rust no duplica el contenido del Heap; solo copia la "tarjeta/puntero" en el Stack y **destruye** la variable original (`s1`).
- **Resultado:** La propiedad del dato se **mueve** a la nueva variable. La variable vieja queda totalmente inválida para proteger el programa.

```
let s1 = String::from("hola"); // Datos en el Heap
let s2 = s1; // La propiedad se mueve a s2. s1 queda inválida.

// println!("{}", s1);
// ❌ ERROR de compilación: No puedes usar s1 porque su valor se movió.
```

Funciones

Punto de Entrada y Definición de Funciones

```
fn bienvenida(nombre: &str) { // función auxiliar
    println!("¡Hola, {}!", nombre);
}

fn main() { // punto de entrada obligatorio
    bienvenida("Rust");
}
```

- `fn` declara una función; el nombre sigue convención *snake_case*.
- `main` es el único identificador reservado como punto de arranque.
- Las funciones pueden definirse antes o después de `main`; el orden no importa.
- Se invocan escribiendo su nombre seguido de `()`.

Parámetros y Valores de Retorno

```
fn sumar(a: i32, b: i32) -> i32 {  
    a + b // expresión sin `;` → valor de retorno implícito  
}  
  
fn main() {  
    let resultado = sumar(3, 5);  
    println!("{}", resultado); // 8  
}
```

- Cada parámetro requiere nombre y tipo separados por `:` — Rust no los infiere.
- `->` seguido del tipo declara qué devuelve la función.
- La última **expresión** sin `;` es el retorno implícito; agregar `;` la convierte en enunciado y causa un error de compilación.
- `return valor;` permite retornos explícitos o anticipados.

Programa Completo

```
fn cuadrado(n: i32) -> i32 {
    n * n
}

fn mensaje(nombre: &str) -> String {
    format!("¡Hola, {}!", nombre)
}

fn main() {
    println!("{}", mensaje("Rust")); // ¡Hola, Rust!
    println!("4² = {}", cuadrado(4)); // 4² = 16
}
```

1. `main` orquesta el flujo general.
2. Las funciones auxiliares encapsulan tareas reutilizables.
3. Los valores fluyen mediante parámetros (entrada) y retornos (salida).

Memory Safety

Ownership

1. Todos los programas tienen que gestionar la forma en que utilizan la memoria de la computadora mientras se ejecutan.
 - Algunos lenguajes tienen un recolector de basura (*garbage collection*) que busca regularmente la memoria que ya no se utiliza a medida que el programa corre.
 - En otros lenguajes, el programador debe reservar y liberar la memoria de forma explícita.
2. Rust utiliza un tercer enfoque: la memoria se gestiona a través de un conjunto de reglas que el compilador verifica. Si se viola cualquiera de estas reglas, el programa no compilará.

El sistema de **Ownership** es un conjunto de reglas que gobiernan cómo un programa en Rust gestiona la memoria:

1. Cada valor tiene un propietario (*owner*).
2. Solo puede haber un propietario a la vez.
3. Cuando el propietario sale del ámbito (*scope*), el valor se destruye (*dropped*).

El *Ownership* (propiedad) define quién es responsable de la información en memoria, lo que naturalmente conlleva la capacidad de transferir esta responsabilidad. La propiedad de la información se transfiere a través de:

- **Asociaciones** (ej. `y = x;`)
- **Asignaciones** (ej. `let y = x;`)
- **Paso de parámetros** (enviar variables a una función)
- **Retornos** (devolver valores desde una función)

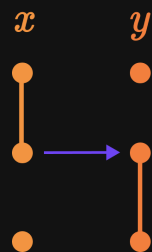
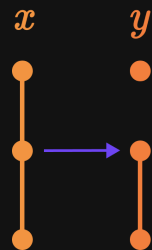
El comportamiento de los datos ante estos fenómenos variará dependiendo de dónde se almacene la información:

- **Si el dato vive en el Stack:** El valor se **duplica automáticamente**. Ambas variables se quedan con una copia independiente y siguen vivas.
- **Si el dato vive en el Heap:** Se realiza una **Transferencia (Move)**. La propiedad se mueve al nuevo destino y la variable original se destruye de inmediato.

Asociaciones y Asignaciones

```
let x = 10;  
let y = x;
```

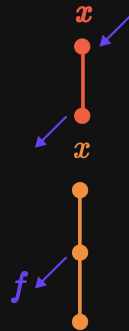
```
let x = String::from("hola");  
let y = x;
```



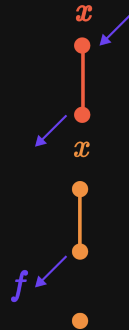
En Rust, las "asociaciones" tras una declaración inicial y las "asignaciones" propiamente dichas a variables mutables siguen exactamente la misma semántica de copia o movimiento).

Paso de Parámetros

```
fn foo(x: i32) {  
    // ...  
}  
  
fn main() {  
    let x = 27;  
    foo(x);  
    println!("{x}");  
}
```

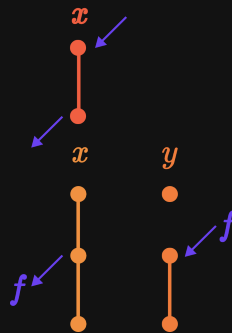


```
fn foo(x: String) {  
    // ...  
}  
  
fn main() {  
    let x = String::from("hola");  
    foo(x);  
    println!("{x}");  
}
```

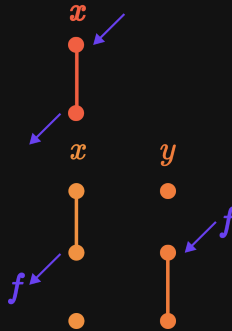


Retornos

```
fn foo(x: i32) {  
    x  
}  
  
fn main() {  
    let x = 27;  
    let y = foo(x);  
    println!("{x}");  
}
```

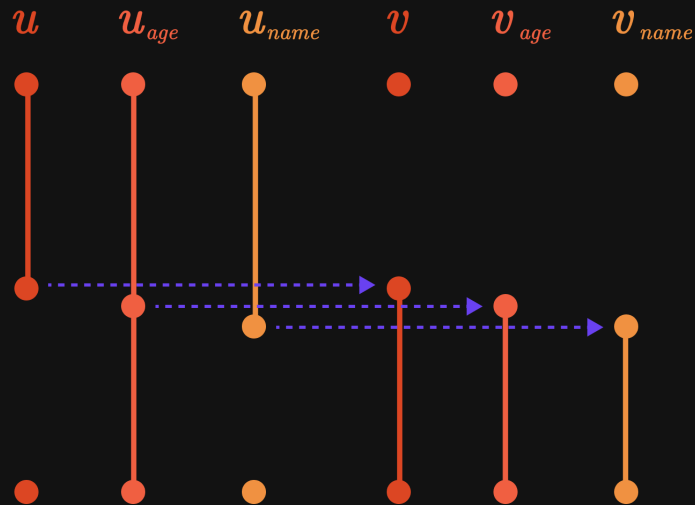


```
fn foo(x: String) {  
    x  
}  
  
fn main() {  
    let x = String::from("hola");  
    let y = foo(x);  
    println!("{y}");  
}
```



Structs

```
struct User {  
  name: String,  
  age: u8,  
}  
  
fn main() {  
  let u = User {  
    name: String::from("Giovanny"),  
    age: 25,  
  };  
  
  let v = u;  
}
```



El Costo del Ownership Estricto

El sistema de *ownership* garantiza seguridad, pero introduce una limitación práctica: **pasar datos del Heap a una función los consume**. Devolver el valor en cada función para recuperarlo es tedioso y no escala. Rust resuelve esto con un mecanismo complementario: el **Borrowing**.

Borrowing y Lifetimes

1. Para flexibilizar el sistema de *ownership*, Rust implementa un mecanismo complementario llamado ***borrow checker***. Este sistema establece reglas sobre cómo tu programa puede acceder a los datos, especialmente cuando múltiples partes necesitan leer o modificar la misma información.
2. Esto previene errores complejos (*bugs*) cuando diferentes secciones del programa intentan manipular los mismos datos de forma simultánea.

Referencias

En Rust, una **referencia** (`&`) es un puntero que permite acceder a un dato sin tomar su propiedad (*ownership*).

- **Préstamo:** El dueño original conserva el control del dato; solo le da un permiso temporal a otra variable.
- **Sintaxis:** Se utiliza el operador `&` para crear una referencia de lectura, y `&mut` para una referencia que permita modificar el dato.

Reglas del borrow checker

Para asegurar que estos préstamos temporales no corrompan la memoria ni causen problemas al compilar, el *borrow checker* vigila estrictamente dos condiciones:

1. Para un instante de tiempo dado, está permitida una y sólo una referencia mutable ó cualquier cantidad de referencias inmutables.
2. Toda referencia debe ser siempre válida.

Válido: Muchas lecturas

```
fn main() {  
    let x = String::from("hola");  
    let r1 = &x;  
    let r2 = &x;  
    println!("{r1} y {r2}");  
}
```

Error: Mezcla de lectura y escritura

```
fn main() {  
    let mut x = String::from("hola");  
    let r1 = &x;  
    let r2 = &mut x;  
    println!("{r1}");  
}
```

Puedes tener infinitas lecturas, pero si creas una referencia mutable (`&mut`), no puede existir nadie más accediendo a ese dato al mismo tiempo.

Tipos de lifetimes

Lifetimes concretos L

- Son tiempos de vida reales y físicos de las variables en memoria.
- Están determinados por el lugar donde se crean las variables y los brackets que marcan su destrucción.
- Son estáticos y ocurren al ejecutar el bloque de código.

Lifetimes genéricos $'a$

- Son variables abstractas que funcionan como parámetros de tiempo.
- No representan un tiempo fijo, sino una relación o un "cupó".

1. El *borrow checker* evalúa constantemente que los **lifetimes concretos** (L) de las referencias se ajusten de forma segura a los límites marcados por los **lifetimes genéricos** ($'a$).
2. En términos simples: vigila estrictamente que ninguna referencia viva más tiempo que el dueño original de los datos, evitando así punteros colgados (*dangling pointers*).

¿Por qué se necesitan los lifetimes genéricos?

1. **Relación de supervivencia:** Son necesarios para indicarle al compilador qué vínculo tienen las referencias entre sí cuando no puede deducirlo por su cuenta.
2. **Garantía de seguridad:** Se aplican para asegurar que un dato retornado (o guardado en una `struct`) nunca viva más tiempo que la referencia original que le dio origen.

Ecuación general de los lifetimes

Sean L y $'a$ lifetimes, s y t referencias de entrada y r la referencia de salida en una función. El borrow checker evalúa que:

$$\max(L(r)) \leq 'a \mid 'a = \min(L(s), L(t))$$

para determinar que toda referencia debe ser siempre válida durante el flujo del programa.

```
fn foo(a: &str, b: &str) -> &str {
    if a.len() >= b.len() {a} else {b}
}

fn main() {
    let s = String::from("Hello world in english!");
    let t = String::from("Hola mundo en español!");
    let r = foo(&s, &t);
    println!("{}", r);
}
```

```
error[E0106]: missing lifetime specifier
--> src/main.rs:1:29
|
1 | fn foo(a: &str, b: &str) -> &str {
|           ----      ----      ^ expected named lifetime parameter
|
= help: this function's return type contains a borrowed value,
but the signature does not say whether it is borrowed from `a` or `b`
help: consider introducing a named lifetime parameter
```

```
fn foo<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() {a} else {b}
}

fn main() {
    let s = String::from("Hello world in english!");
    let t = String::from("Hola mundo en español!");
    let r = foo(&s, &t);
    println!("{}", r);
}
```

```
error[E0106]: missing lifetime specifier
--> src/main.rs:1:29
|
1 | fn foo(a: &str, b: &str) -> &str {
|           ----      ----      ^ expected named lifetime parameter
|
= help: this function's return type contains a borrowed value,
but the signature does not say whether it is borrowed from `a` or `b`
help: consider introducing a named lifetime parameter
```



```
struct User {  
    age: i32,  
    name: & str  
}  
  
fn main() {  
    let n = "Giovanny";  
    let usr = User {  
        age: 32,  
        name: n  
    };  
    println!("Mi nombre es {} y tengo {} años", usr.name, usr.age);  
}
```

```
error[E0106]: missing lifetime specifier  
--> src/main.rs:3:11  
   |  
3 |     name: & str  
   |           ^ expected named lifetime parameter  
help: consider introducing a named lifetime parameter
```

```
struct User<'a> {  
    age: i32,  
    name: &'a str  
}  
  
fn main() {  
    let n = "Giovanny";  
    let usr = User {  
        age: 32,  
        name: n  
    };  
    println!("Mi nombre es {} y tengo {} años", usr.name, usr.age);  
}
```

```
error[E0106]: missing lifetime specifier  
--> src/main.rs:3:11  
   |  
3 |     name: & str  
   |           ^ expected named lifetime parameter  
   |  
help: consider introducing a named lifetime parameter
```


Evaluación de las referencias

```
fn foo<'a>(a: &'a str, b: &'a str) -> &'a str {  
    if a.len() >= b.len() {a} else {b}  
}  
  
fn main() {  
    let s = String::from("Hello world in english!");  
    let t = String::from("Hola mundo en español!");  
    let r = foo(&s, &t);  
    println!("{}", r);  
}
```

$$'a = \min(L(s), L(t))$$

Sustituyendo los tiempos de vida reales:

$$L(s) = L(t) = L(r)$$

$$'a = L(s)$$

Condición del *borrow checker*:

$$\max(L(r)) \leq 'a$$

$$L(r) \leq L(s) \Rightarrow \text{Válido}$$

```

fn foo<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() {a} else {b}
}

fn main() {
    let r: &str;
    let s = String::from("Hello world in english!");
    {
        let t = String::from("Hola mundo en español!");
        r = foo(&s, &t);
    }
    println!("{}", r);
}

```

$$'a = \min(L(s), L(t))$$

Como t vive en un bloque interno más pequeño:

$$L(t) < L(s) \Rightarrow 'a = L(t)$$

Condición del *borrow checker*:

$$\max(L(r)) \leq 'a$$

$$L(r) \leq L(t) \Rightarrow \text{FALSO}$$

Seguridad de Memoria en Rust

Al trabajar en conjunto, el sistema de **Ownership**, el **Borrow Checker** y las reglas de **Lifetimes** eliminan los errores de gestión de memoria más comunes de la industria:

- **Punteros nulos:** Rust garantiza que toda referencia apunte siempre a un lugar válido en memoria.
- **Desbordamientos de búfer:** Valida estrictamente que las lecturas y escrituras ocurran dentro de los límites de los arreglos.
- **Punteros colgados / Doble liberación:** Erradicados por completo gracias al control estricto de ámbitos de vida.

- **Carreras de datos (Data races):** Impide el acceso simultáneo y desordenado a un mismo dato en entornos multihilo.
- **Comportamientos indefinidos:** El compilador rechaza cualquier zona gris de la memoria (*undefined behavior*).
- **Cero Garbage Collector:** No se necesita pausar el programa en ejecución; todo se resuelve al compilar.

Conclusión: Rust traslada la seguridad de la memoria al **tiempo de compilación**. Si tu código compila, es matemáticamente seguro en memoria.